

Computer Science & IT

Database Management System



Transactions & concurrency control

Lecture No. 05



By- Vishal Sir



Recap of Previous Lecture

- ✓ Topic WW problem
- ✓ Topic Lost update problem
- ✓ Topic Classification of schedules
- ✓ Topic Classification based on recoverability



Topics to be Covered



✓ {
Topic

Classification based on recoverability

✓ {
Topic

Classification based on serializability

Topic

Conflicting and non-conflicting operations

Topic

Conflict serializable schedule

Topic

Precedence graph

Topic

Topological order



Topic : Classification of schedule

Classification based on Recoverability

- ① Irrecoverable Schedule
- ② Recoverable Schedule
- ③ Cascadeless Rollback Recoverable Schedule
- ④ Strict Recoverable Schedule

Classification based on Serializability

- ① Conflict Serializable Schedule
- ② View Serializable Schedule

Classification based on
Recoverability

Note :-

If transaction T_j reads the value written by an uncommitted transaction T_i , then transaction T_j is dependent on transaction T_i .

If transaction T_i rollback for some reason then we will have to rollback all the transactions that are dependent on transaction T_i { for our example it is T_j }

T_i	T_j
$W_i(A)$	
...	
	$R_j(A)$

T_j reads the value written by an uncommitted transaction T_i
{ T_j is dependent on T_i }

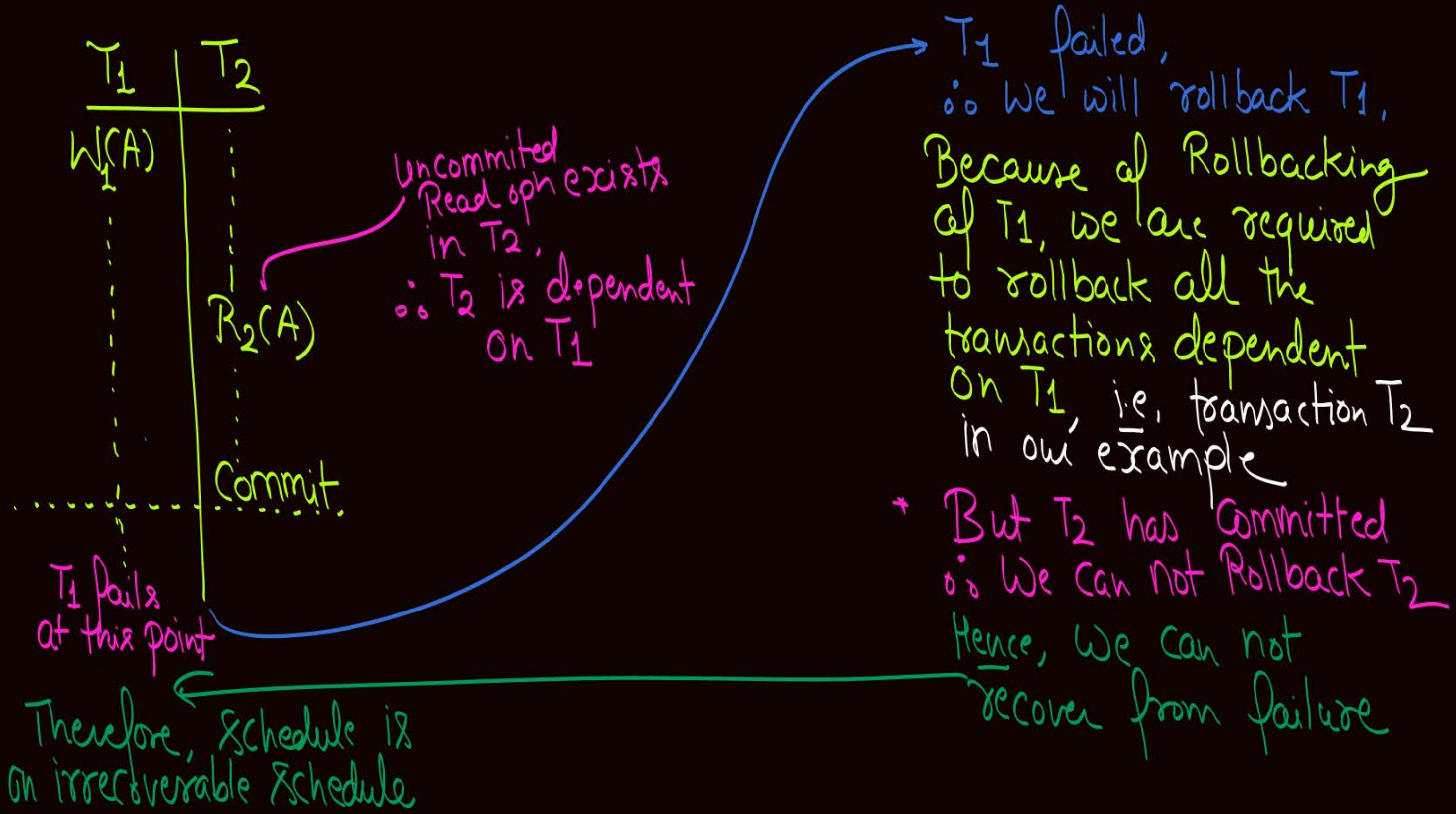
{ We can rollback a transaction only if that transaction has not yet committed, Rollbacking of a committed transaction is not allowed }



Topic : Irrecoverable schedule

- * It is not possible to rollback a Committed transaction.
- * If there arise a situation when we are required to rollback a Committed transaction, then it will not be possible to rollback that transaction, and hence we will not be able to recover from that failure. Such Schedules are called irrecoverable schedule.

Irrecoverable Schedule



Irrecoverable Schedule



uncommitted
opⁿ

$R_2(A)$

T_2 is dependent
on T_1

No transaction has performed
the commit opⁿ.

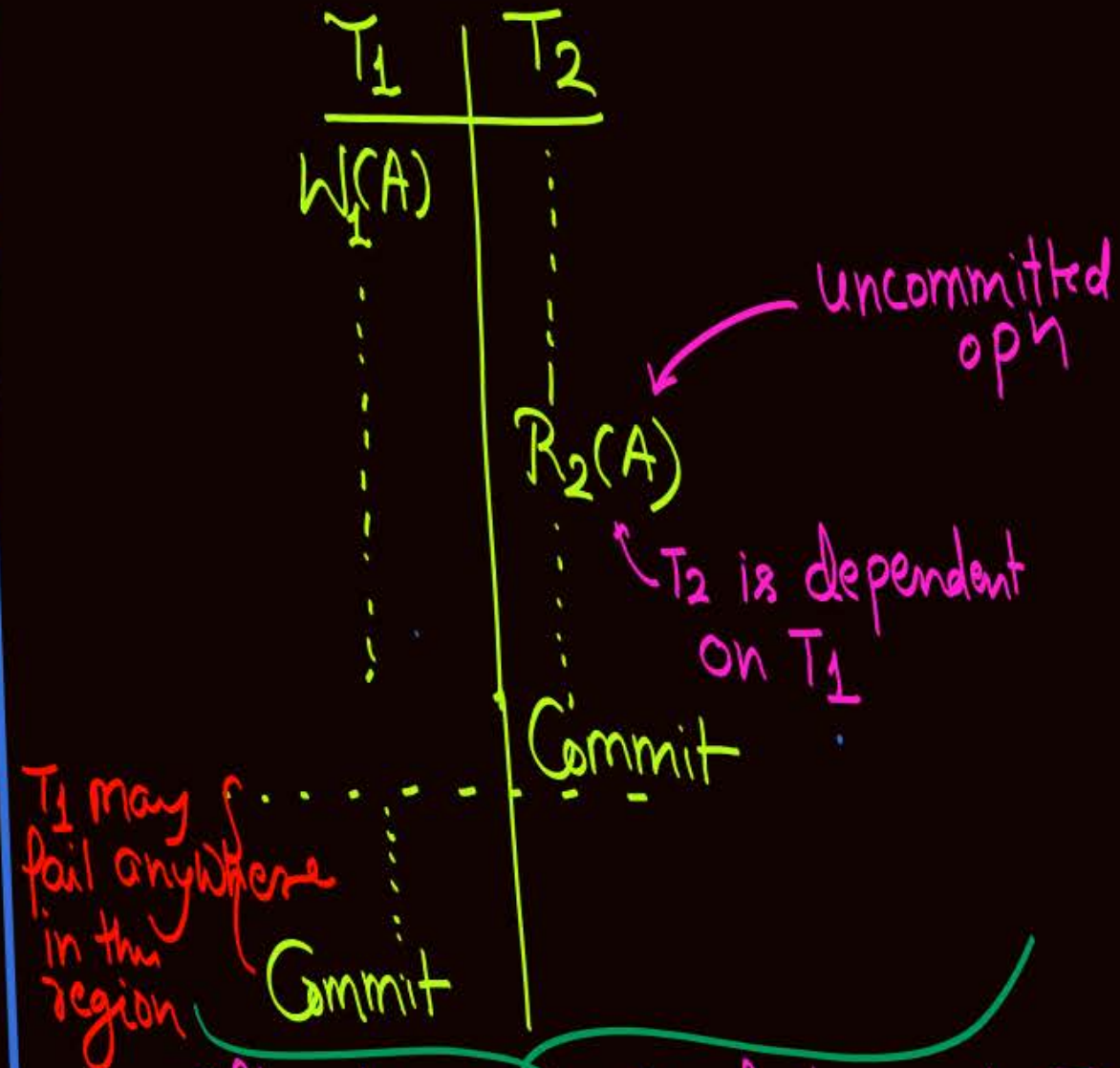
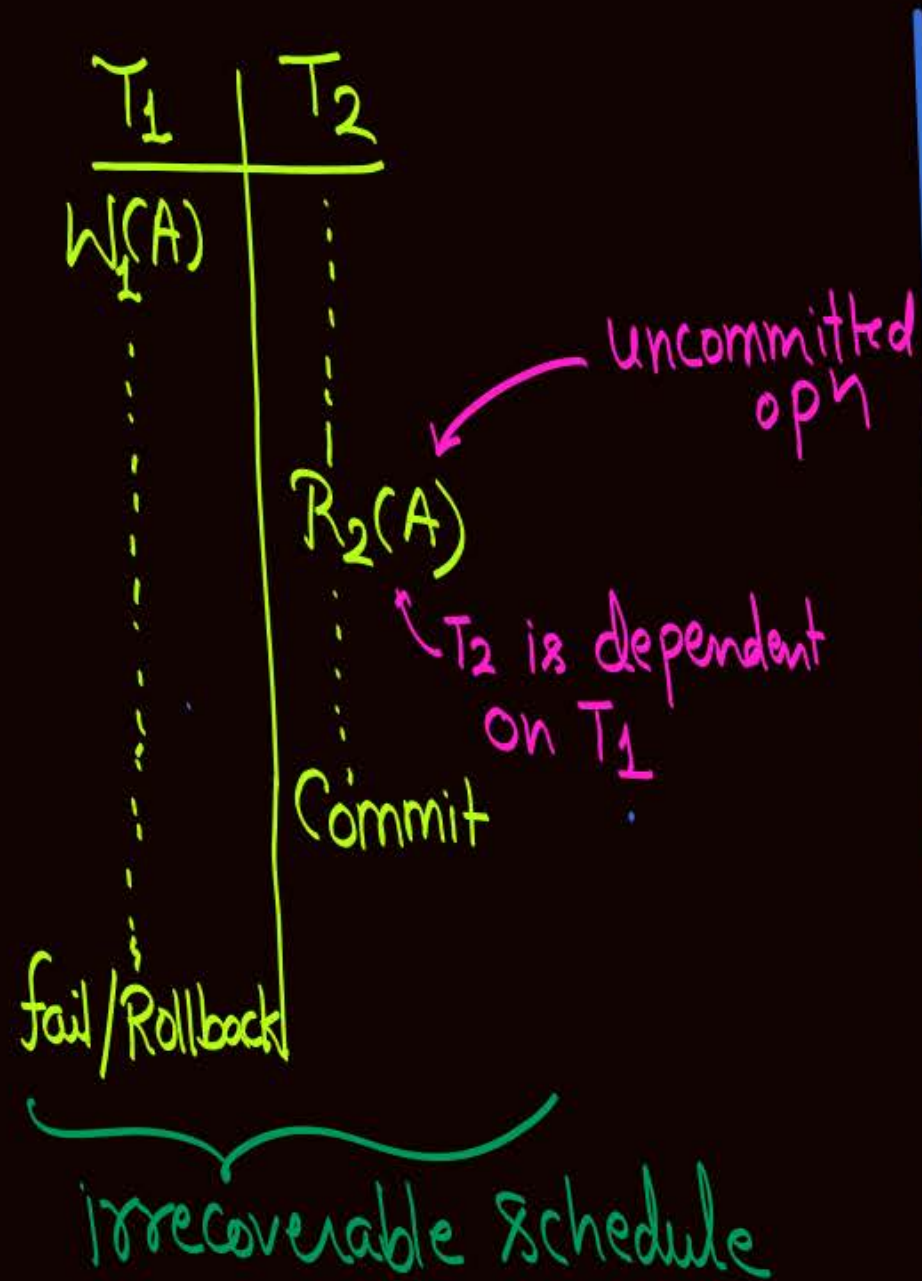
∴ If failure occur then
both can rollback

ie, we will be able to recover
from failure

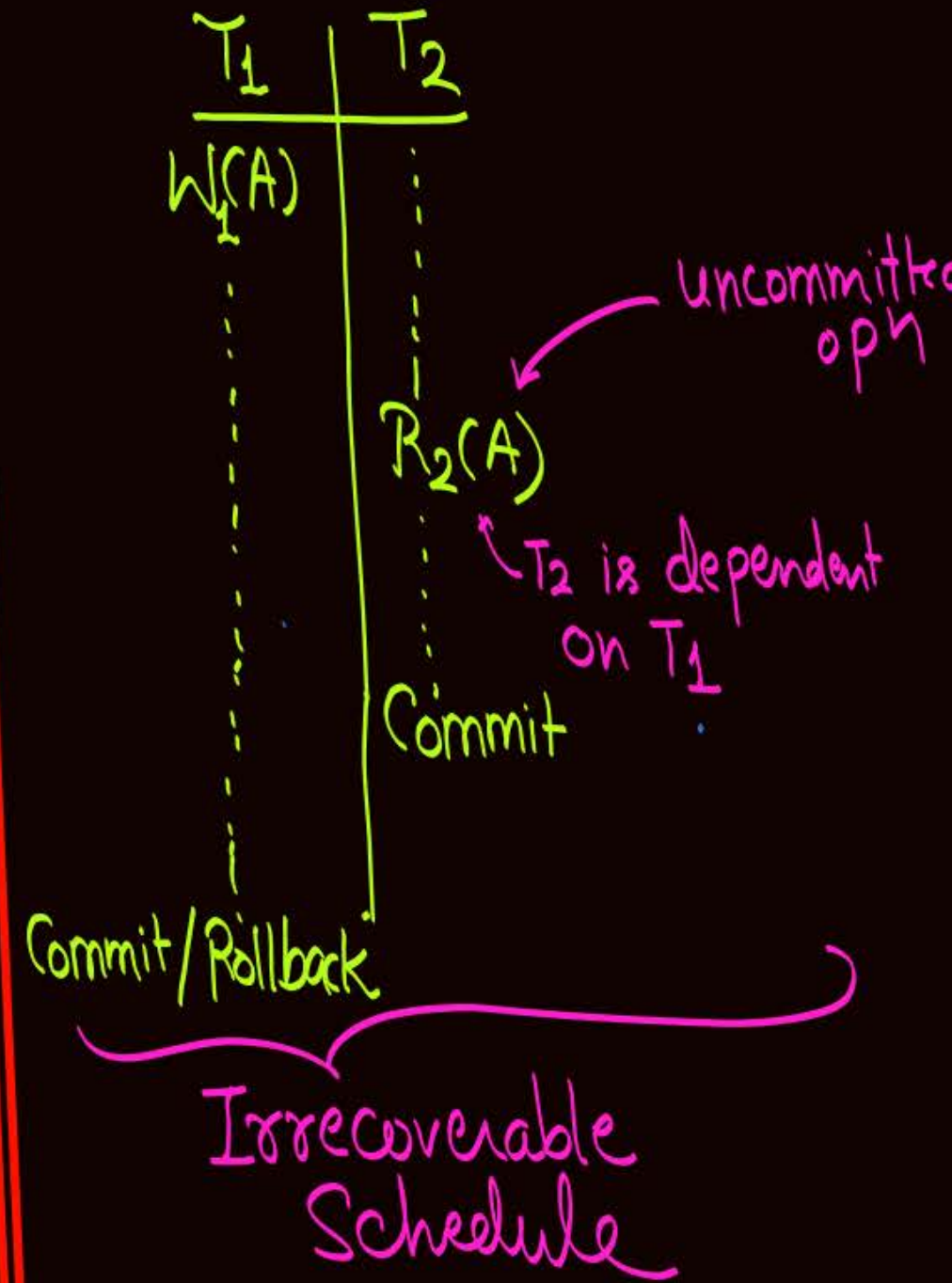
∴ Schedule is a

Recoverable Schedule

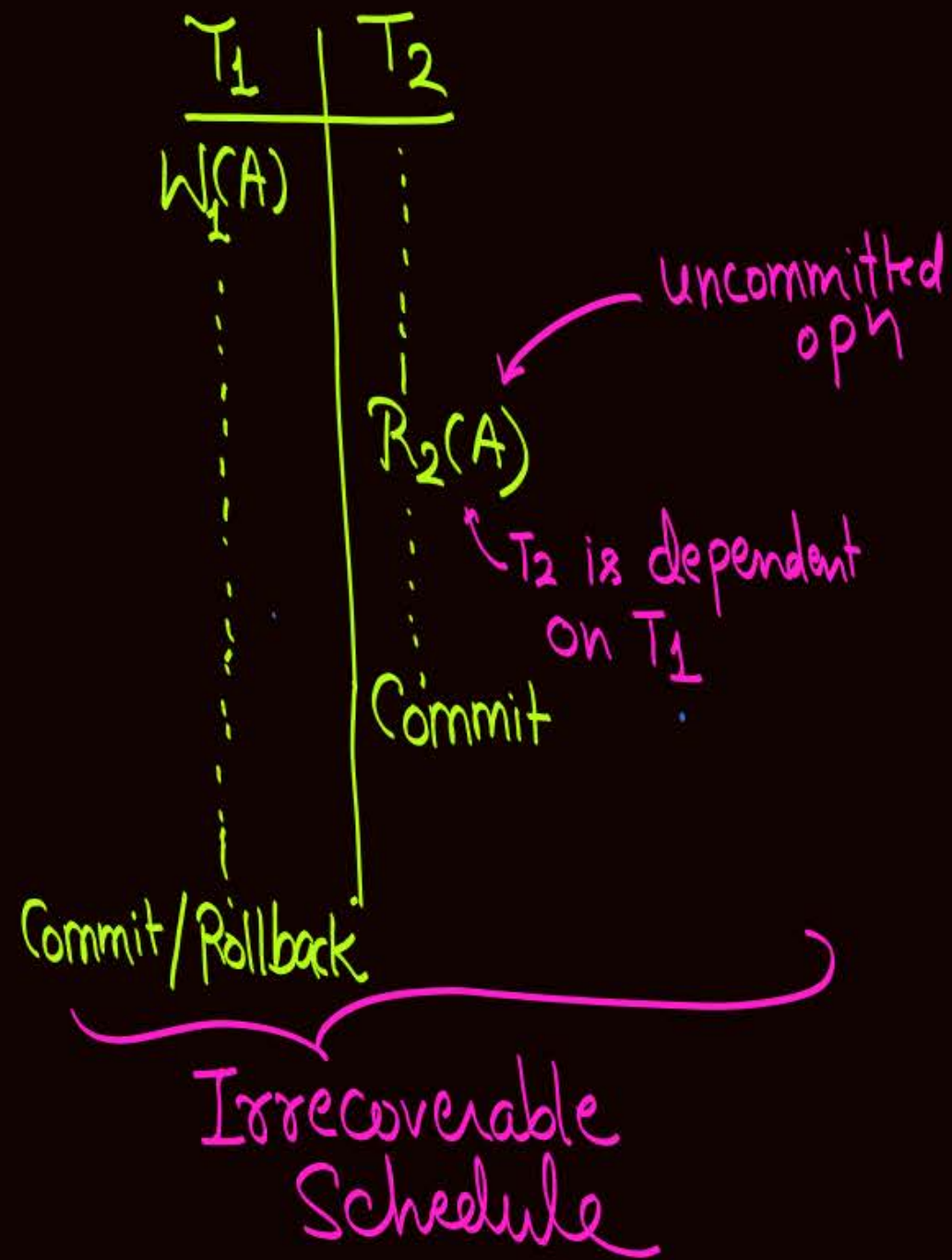
Irrecoverable Schedule



After the Commit of transaction T₂, schedule is unsafe because T₁ may fail, and if it happens we will not be able to recover
∴ it is also irrecoverable schedule



Irrecoverable Schedule



Let two transaction T_1 and T_2 exists in a schedule and

- If transaction T_2 is dependent on transaction T_1 , and if transaction T_2 (i.e. dependent transaction) Commit before Commit or Rollback of transaction T_1 (i.e. the transaction on which it depends), then that schedule is called irrecoverable schedule.

Irrecoverable Schedule

- Uncommitted Read opⁿ are Necessary (not sufficient) for a schedule to be irrecoverable

• i.e, ① If there is no uncommitted read opⁿ in the schedule then schedule can not be irrecoverable {i.e, it is recoverable}

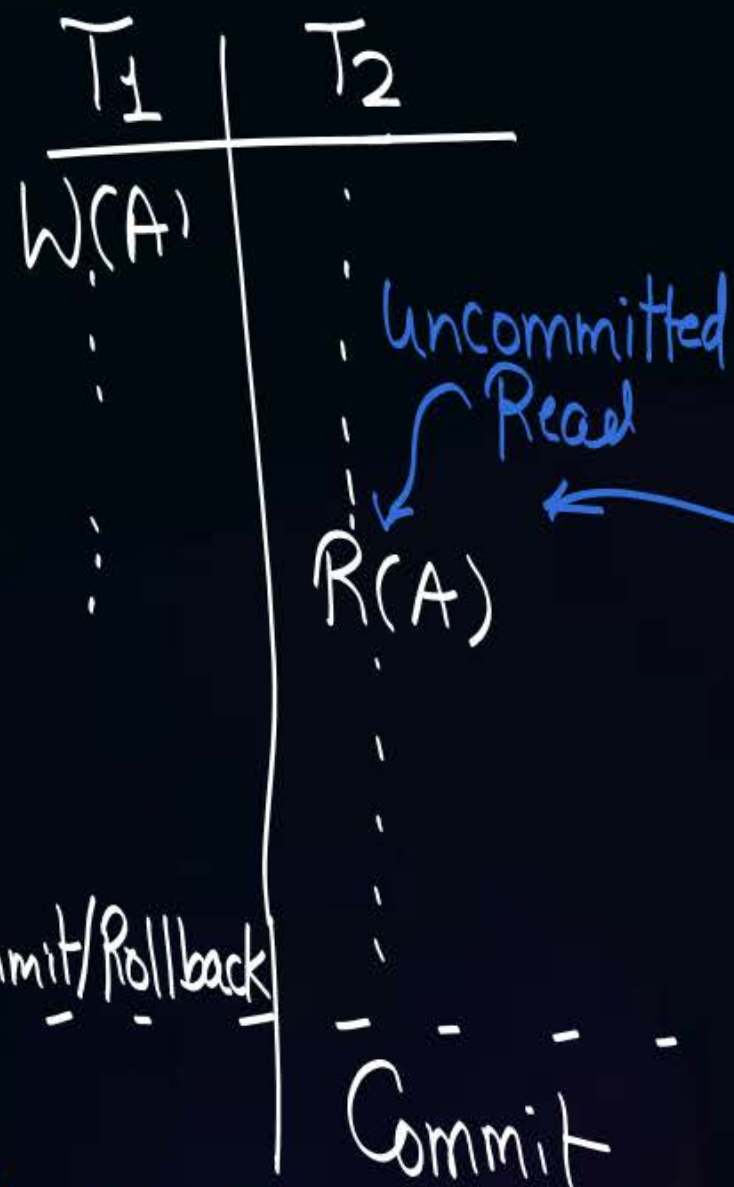
Ans ② If uncommitted opⁿ exists in the schedule, then schedule may or may not be irrecoverable.



Topic : Recoverable schedule

For a schedule to be called a recoverable schedule either uncommitted Read opⁿ should not exist in the schedule

(or) if uncommitted read opⁿ exist in the schedule, then dependent transaction must commit after the Commit/Rollback of the transactions on which they depend



Recoverable Schedule



Topic : Cascading rollback problem

→ If failure of one transaction results in rollbacking of a set of other transactions along with rollbacking of failed transaction, then we say that cascading rollback problem exists in the schedule



Topic : Cascading rollback problem

Cascading Rollback problem results in wastage of CPU time & IO time



T ₁	T ₂	T ₃	T ₄	T ₅
W(A)	R(A)	R(A) W(A)	R(A)	R(A)

* T₁ fails

No one has committed
∴ All can rollback

Hence Recoverable Schedule
let T₁ fails at this point

- T₁ fails, ∴ We will rollback T₁
- Because of Rollbacking of T₁ we need to rollback all the transactions dependent on T₁, i.e., T₂ & T₃
- Because of rollbacking of T₂ & T₃ We need to rollback the transactions dependent on T₂ & T₃. No transaction is dependent on T₂, but T₄ & T₅ are dependent on T₃, ∴ We need to rollback T₄ & T₅ as well.

Because of Rollback of one transaction we had to rollback multiple transaction,
∴ Cascading Rollback Problem exist



Topic : Cascadeless recoverable schedule

→ If uncommitted Read opⁿ exists in the schedule, then dependency among the transactions will exist, and if dependency exists among the transactions, then because of failure of one transaction we will have to rollback all dependent transactions and hence cascading rollback problem will exist.

∴ For Cascadeless recoverable schedule, uncommitted "read" operations must not be allowed.



Topic : Cascadeless recoverable schedule

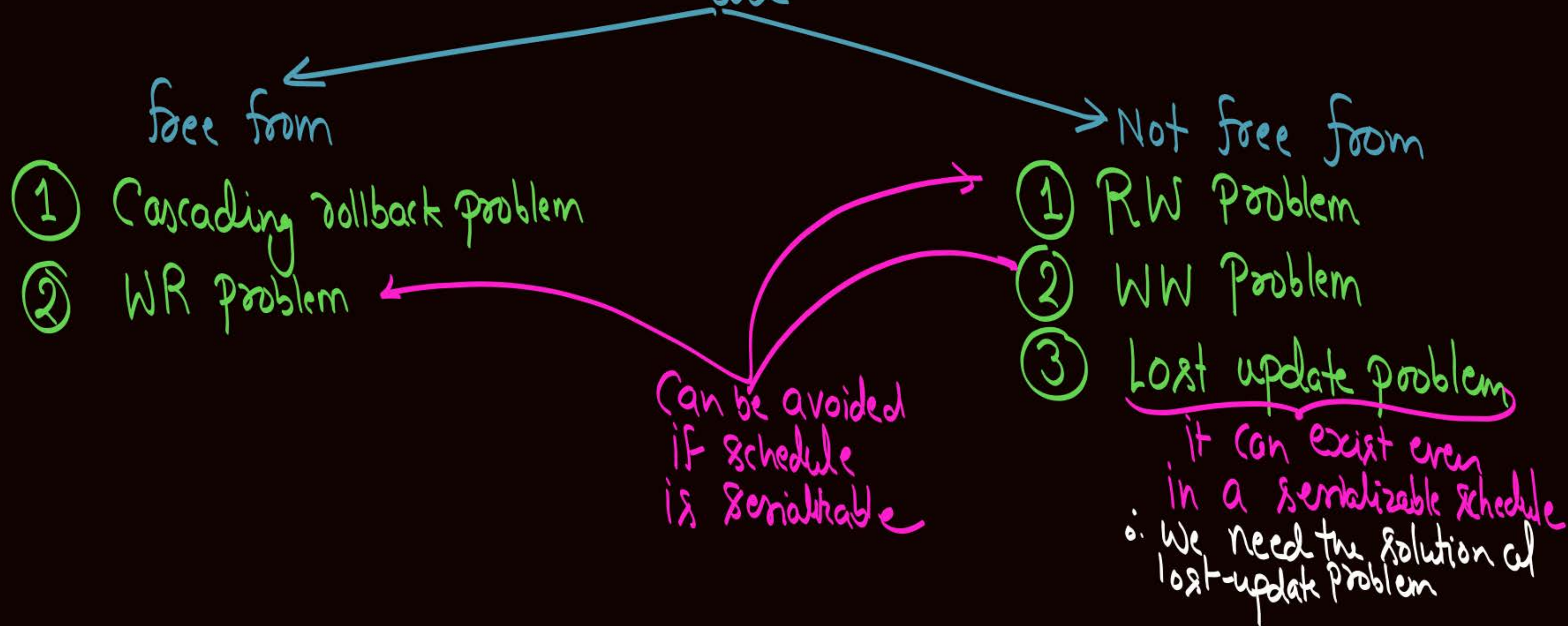
→ For cascadeless recoverable schedule uncommitted read opⁿ must not exist.

T ₁	T ₂
W(A)	
⋮	
Commit/Rollback	
	R(A)

For cascadeless recoverable schedule, if transaction T₁ updates data item 'A' then no other transaction should be allowed to read the value of data item A updated by transaction T₁ until commit or rollback of transaction T₁.

→ Cascading rollback problem can exist in any type of schedule, i.e., serializable as well as non-serializable schedules

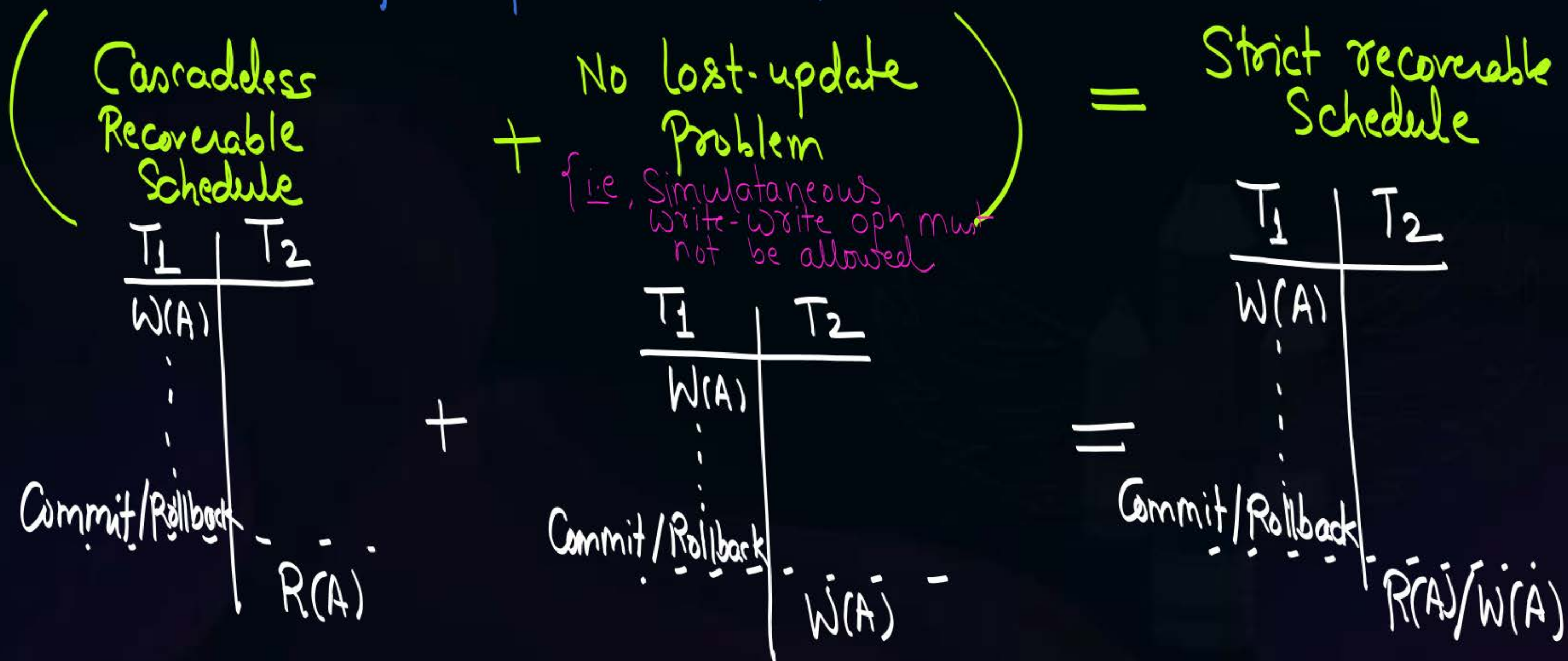
Cascadeless recoverable Schedules





Topic : Strict recoverable schedule

Strict recoverable schedule is Cascadeless recoverable schedule as well as free from lost-update problem





Topic : Strict recoverable schedule

for a schedule to be strict recoverable schedule
If any transaction T_i updates the dataitem 'A'
then no other transaction should be allowed
to read the dataitem or write the dataitem 'A'
until the commit or rollback of transaction T_i

T_1	T_2
$W(A)$	
⋮	
Commit/Rollback	
	$R(A)/W(A)$

Strict recoverable Schedule

Ans.

free from

Not free from

- ① Cascading Rollback problem
- ② Lost update problem
- ③ W-R problem
- ④ WW problem

① R-W problem

Can be solved if the
Schedule is serializable
Schedule

Criteria for Consistency: -

- ① Schedule must be strict recoverable
- ② Schedule must be serializable schedule

No cascading Rollback
&
No lost update

No WR, WW or RW
Problem



Topic : Classification of schedule

Classification based on Recoverability

- ① Irrecoverable Schedule
- ② Recoverable Schedule
- ③ Cascadeless Rollback Recoverable Schedule
- ④ Strict Recoverable Schedule

Already done

Classification based on Serializability

- ① Conflict Serializable Schedule
- ② View Serializable Schedule

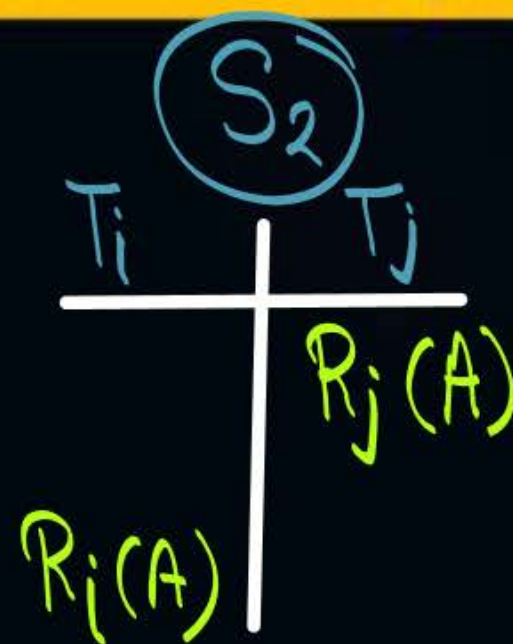
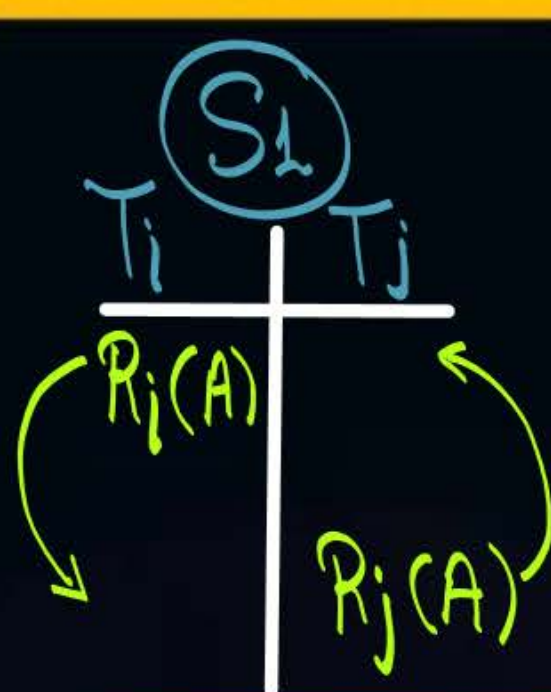
Classification based on Serializability

Conflict Serializable Schedules
(C.S.S.)



Topic : Conflicting and non-conflicting operations

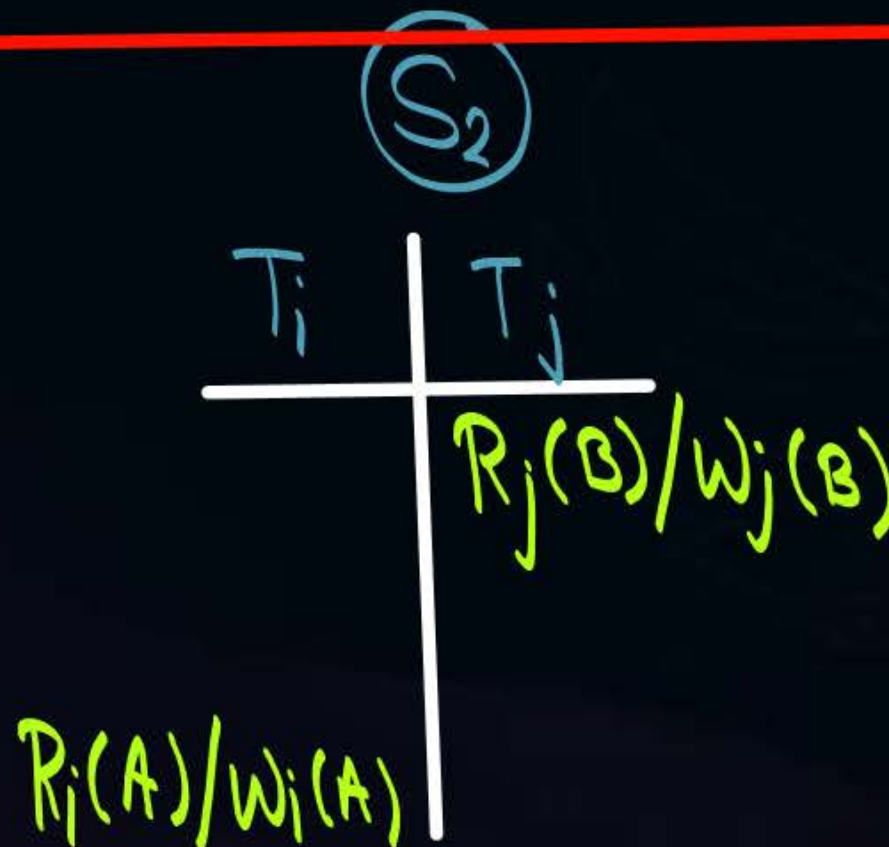
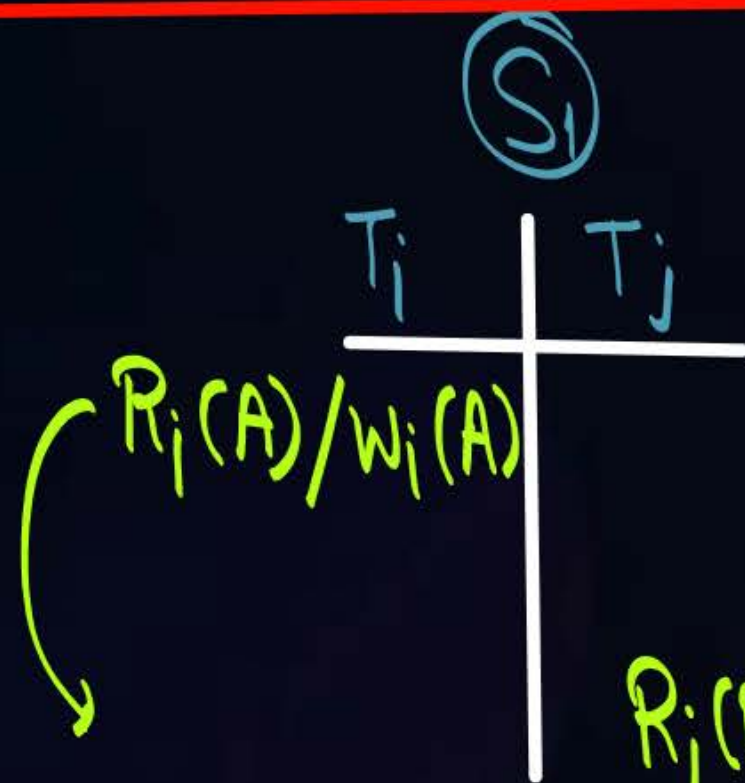
①



$$S_1 \equiv S_2$$

∴ $R_i(A)$ & $R_j(A)$ are not Conflicting opⁿ

②



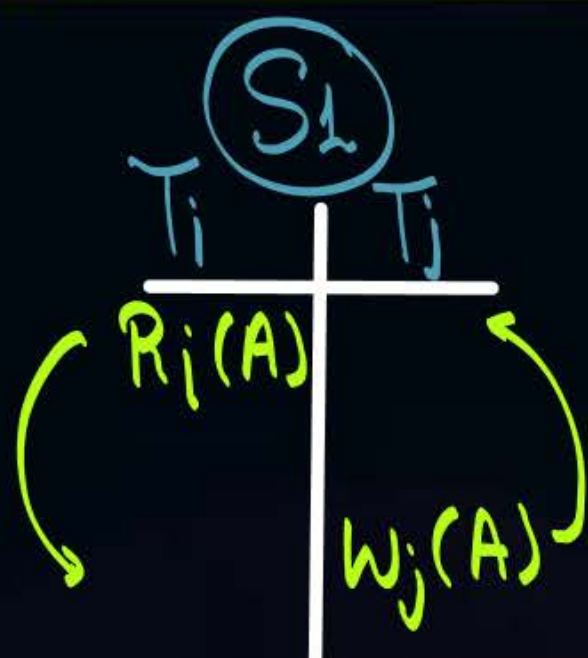
$$S_1 \equiv S_2$$

∴ $R_i(A)/W_i(A)$ is non-conflicting with $R_j(B)/W_j(B)$



Topic : Conflicting and non-conflicting operations

③



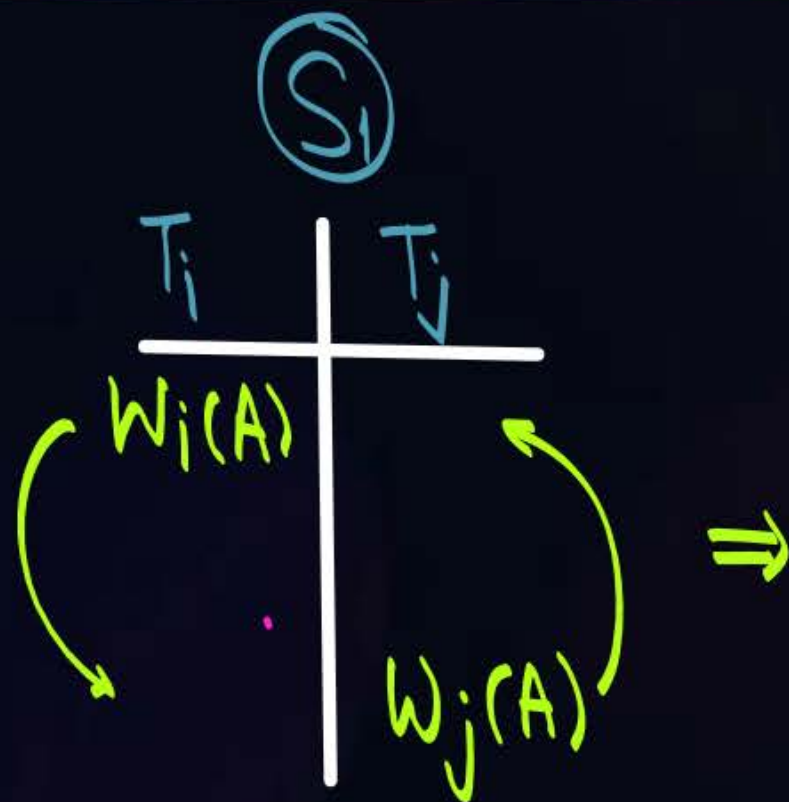
$\Rightarrow$



$$S_1 \neq S_2$$

$\therefore R_i(A) \& W_j(A)$
are conflicting
opn

④



$\Rightarrow$



$$S_1 \neq S_2$$

$\therefore W_i(A) \& W_j(A)$
are conflicting
opn.



Topic : Conflicting and non-conflicting operations

✓ **Conflicting Operations** : Two operations are called as conflicting operations if all the following conditions hold true for them -

- ✓ ☐ Both the operations belong to different transactions
- ✓ ☐ Both the operations are on the same data item
- ✓ ☐ At least one of the two operations is a write operation



Topic : Conflicting and non-conflicting operations

- **Non-Conflicting Operations** : Two operations are non-conflicting operations only if
- ✓ ☒ Both the operations are on different data items
 - or
 - ✓ ☒ Both the operations are read operations



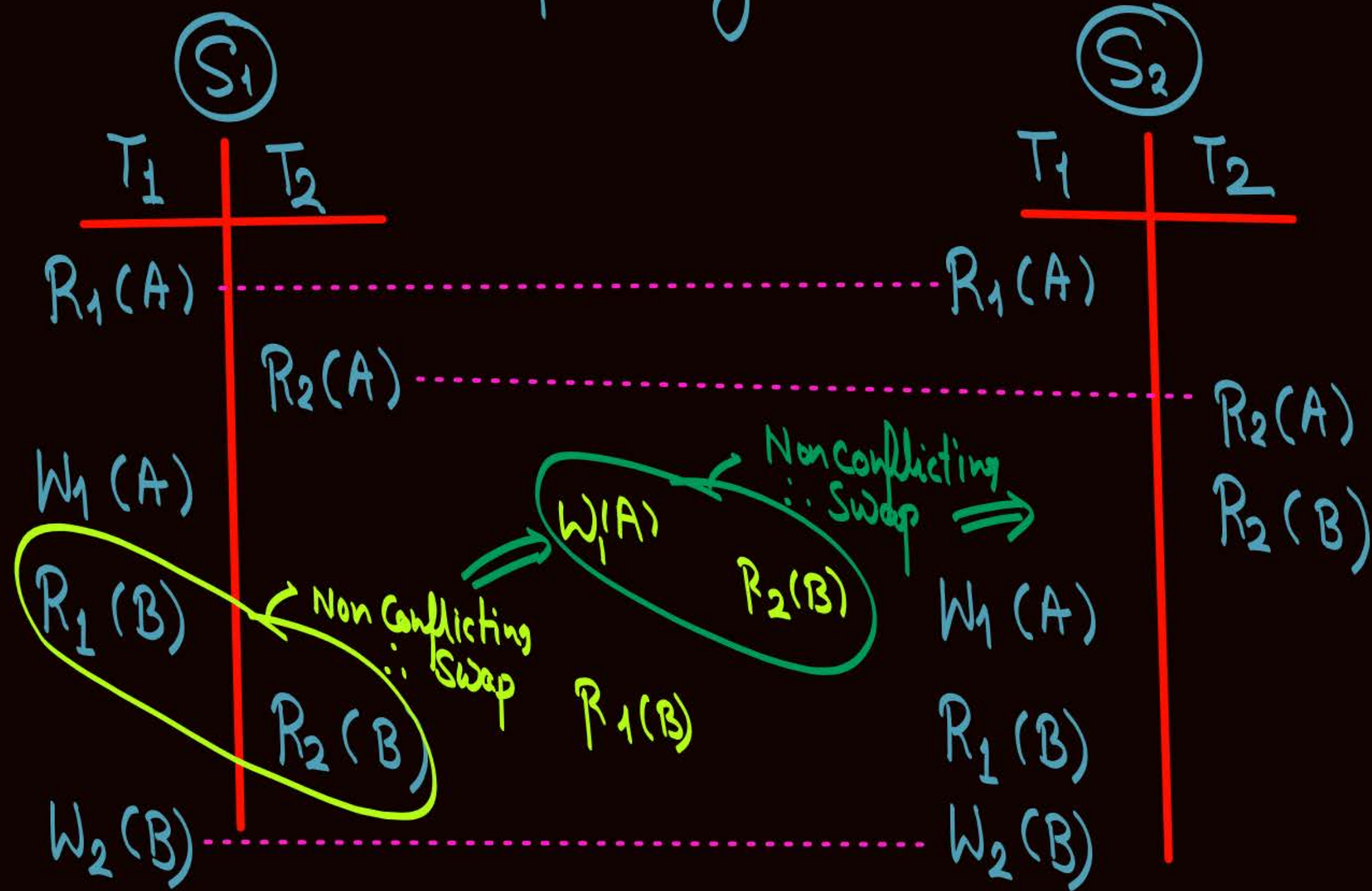
Topic : Conflict equivalent schedule



Consider two schedules S_i and S_j ,

If one can be transformed into another by swapping the position of some of the non-conflicting operations in it, then schedule S_i and S_j are called conflict equivalent schedule.

eg Consider the following Schedules



- S₁ can be converted into S₂ by swapping the position of some of the non-conflicting operations.
- ∴ S₁ is Conflict Equivalent to S₂

eg Consider the following Schedules

(S₁)

T ₁	T ₂
R ₁ (A)	
W ₁ (A)	
	R ₂ (A)
R ₁ (B)	
W ₂ (B)	
	R ₂ (B)

(S₂)

T ₁	T ₂
R ₁ (A)	
W ₁ (A)	
	R ₂ (A)
	R ₂ (B)
R ₁ (B)	
W ₂ (B)	

for Conflictint Equivalence
Swapping of Conflicting
opn is not allowed.

∴ S₁ & S₂ are
not Conflict Equivalent
to each other

Conflicting
Pair of opn
∴ We can
not swap

⇒ Hence
S₁ can not be transformed into S₂



Topic : Conflict serializable schedule

- If a given schedule can be converted into ^{any} serial schedule by swapping the positions of some of its non-conflicting operations, then it is called a conflict serializable schedule.

i.e. If given schedule is conflict equivalent to any of the serial schedule, then schedule is called Conflict serializable schedule.

eg Consider the following Schedules

(S₁)

T ₁	T ₂
R ₁ (A)	
W ₁ (A)	
	R ₂ (A)
R ₁ (B)	
W ₂ (B)	
	R ₂ (B)

Conflicting opⁿ

∴ R₂(A) Can not be moved above W₁(A),
Hence S₁ Can not be converted into
Serial Schedule T₂ → T₁

R₂(A) is non-conflicting with
R₁(B) as well as W₁(B),
∴ we can swap the position
and we can move R₂(A) below
the opⁿ R₁(B) & W₁(B),
and resulting schedule will be
Serial Schedule T₁ → T₂

Schedule S₁ is conflict equivalent to at least one
serial schedule {i.e. T₁ → T₂}, Hence Schedule S₁ is
a Conflict serializable schedule

T₁ → T₂

T ₁	T ₂
R ₁ (A)	
W ₁ (A)	
R ₁ (B)	
W ₁ (B)	
	R ₂ (A)
	R ₂ (B)

T₂ → T₁

T ₁	T ₂
	R ₂ (A)
	R ₂ (B)
R ₁ (A)	
W ₁ (A)	
R ₁ (B)	
W ₁ (B)	



Topic : NOTE



- ① If schedule S is a Conflict serializable schedule, then it is guaranteed that schedule ' S ' is a Serializable schedule
- ② If schedule S is not a Conflict serializable schedule, then schedule S may or may not be a serializable schedule



Topic : NOTE



- ③ Conflicting operations of transactions will define the precedence order in which transactions must appear in any equivalent serial schedule

eg. . .

T_i	T_j
$R_i(A)$	$W_j(A)$

T_i then T_j , i.e., in any equivalent serial schedule T_i must execute before T_j



Topic : Precedence graph

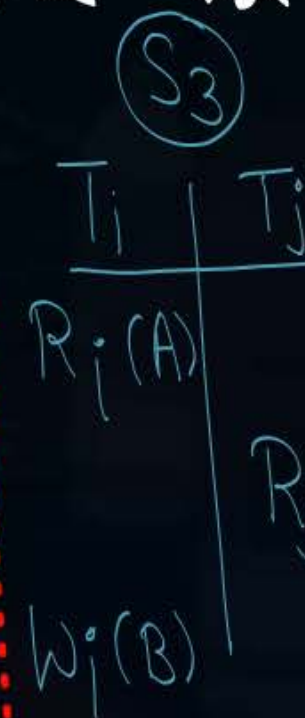
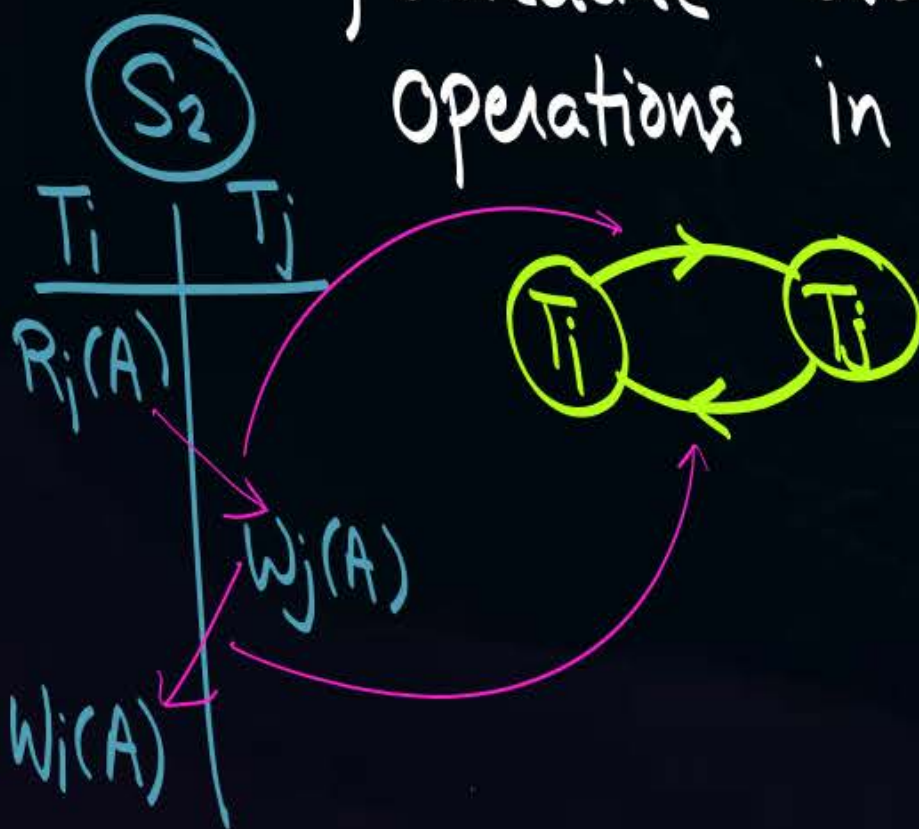
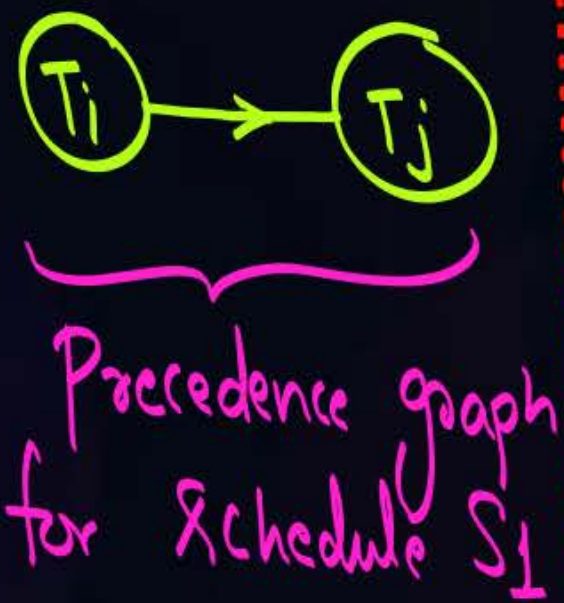
Precedence graph $G=(V,E)$ is a directed graph.

Where : V = Set of vertices = Set of transactions of the schedule

E = Set of edges = Set of directed edges based on the Precedence order defined by Conflicting Operations in the schedule



$\Rightarrow$



No edge in the precedence graph

No pair of conflicting opⁿ in S₃
 $\therefore$ No precedence
Hence No edge



2 mins Summary

Topic

Classification based on recoverability

Topic

Classification based on serializability

Topic

Conflicting and non-conflicting operations

Topic

Conflict serializable schedule

Topic

Precedence graph

Topic

Topological order

THANK - YOU